DECK 02 · WEEK 2

First Deployments

This week you write and deploy Solana code for the first time: a default Anchor deploy, then a PDA-backed counter.

TWO EXERCISES, ONE DEPLOYMENT LOOP

Exercise 1 proves your environment works. Exercise 2 proves you can manage on-chain state with PDAs.

Exercise 1 · Hello Solana

Install and verify tooling, configure devnet, create an Anchor project, build, deploy, run tests, verify on Explorer.

`OUTPUT: DEPLOYED PROGRAM ID`

Exercise 2 · Counter PDA

Write `initialize` + `increment` instructions, derive PDA from seeds, run TS client/test, inspect account and transaction outputs.

`OUTPUT: USER-SCOPED PDA STATE`

AI SUPPORT IS INTENTIONAL

This week includes more guidance than later decks. Use it to form habits: inspect outputs, verify assumptions, and debug from evidence.

TOOLCHAIN CHECKLIST

Before deploying, verify every dependency from one command line.

SETUP VERIFICATION

```
rustc --version && solana --version && anchor --version && surfpool --version && node --version && yarn --version
```

If this command fails, stop and fix installation first. Most week-two errors come from mismatched or missing tools, not program logic.

COMMON AI MISTAKE

Outdated install commands for Anchor are frequent. Confirm install steps against official docs when AI suggests alternatives.

CONFIGURE DEVNET FIRST

Your deployment target and wallet must be explicit before you run Anchor commands.

NETWORK + WALLET SETUP

```
solana config set --url devnet
solana-keygen new
solana airdrop 2
solana config get
```

01 **RPC URL** shows devnet.

02 **Keypair path** points to your wallet.

03 **Balance** is funded with test SOL.

This wallet pays for account creation and deployments in devnet, so keep it funded throughout the week.

FROM TEMPLATE TO ON-CHAIN PROGRAM

The default Anchor scaffold is enough to complete your first real deployment.

FIRST DEPLOY FLOW

```
anchor init hello_solana
cd hello_solana
anchor build
anchor deploy
anchor test
```

What success looks like

`anchor build` completes, `anchor deploy` prints a program ID, and `anchor test` passes end to end.

Explorer verification

Search your program ID on Explorer (devnet). Then open a test transaction and inspect instruction + account details.

USE AI FOR EXPLANATION, NOT BLIND EXECUTION

Installation and generated files are ideal for guided AI walkthroughs.

ASK AI

Diagnose installation failures by pasting full command + OS + complete error text.

ASK AI

Explain generated Anchor files: `Anchor.toml`, `programs/*/lib.rs`, and `tests/`.

ASK AI

Walk through each line of the starter TypeScript test to map calls, accounts, and assertions.

WATCH FOR

Old web3.js v1 syntax and stale Anchor install guidance. Confirm versions before adopting snippets.

ACCOUNTS + PDAS

Build a counter.

INITIALIZE · INCREMENT · VERIFY

DECIDE THE MODEL FIRST

The quality of your AI output depends on clear requirements.

PROGRAM REQUIREMENTS

01 Counter account stores **count** and **authority**.

02 PDA seeds include static seed + user wallet for per-user uniqueness.

03 `initialize` creates PDA account and sets count to 0.

04 `increment` updates existing PDA state.

THINK FIRST

01 Who pays account rent on `initialize`?

02 Who is allowed to call `increment`?

03 What happens on duplicate `initialize`?

04 Will seeds match in program and client code?

SPECIFIC PROMPTS CREATE AUDITABLE CODE

Ask for framework, seeds, instructions, account fields, and tests in one constrained request.

EXAMPLE PROMPT SHAPE

```
Using Anchor in Rust:
- Counter PDA seeds = ["counter", user_pubkey]
- initialize creates account, payer = signer
- increment increases count by 1
- account stores authority: Pubkey + count: u64 (+ bump)
- include TS test: init -> 3 increments -> assert value = 3
```

HABIT

Do not accept first output. Read constraints, space calculation, and seed consistency before running tests.

FOUR AREAS TO CHECK

Most failures come from these mismatches, not from complex Rust syntax.

Account struct + space

Verify discriminator + fields + bump bytes are included in allocation.

Seeds in all locations

`initialize`, `increment`, and TypeScript derivation must use the same seed order and content.

Constraints + signer checks

Look for `init`, `payer`, `seeds`, `bump`, and authority enforcement.

Test quality

Test should assert account values, not only that transactions did not throw.

TESTS PASSING ISN'T THE FINISH LINE

Read the actual on-chain account and compare derivations to prove your model is correct.

INSPECT PDA ACCOUNT

```
solana account <PDA_ADDRESS>
```

Confirm owner = your program ID, data exists, and account length matches your expected layout.

Explorer checks

Open the PDA and recent transactions on devnet Explorer. Inspect instruction accounts and results.

Client derivation check

Compute PDA with `findProgramAddressSync` and compare with on-chain address. If mismatch: seed drift exists.

MAP ERROR TEXT TO ROOT CAUSE

Diagnose first, then ask AI with full context and relevant code.

ERROR	LIKELY CAUSE	ACTION
Account already in use	`initialize` called twice for same PDA.	Expected for duplicate init; test idempotency path.
A seeds constraint was violated	Seed mismatch between program and client.	Normalize seed bytes/order in all calls.
Insufficient funds	Devnet wallet out of SOL.	Run `solana airdrop 2` and retry.
AccountNotInitialized	`increment` before `initialize`.	Enforce init-first workflow in tests/client.
Transaction simulation failed	Wrong account list or missing signer.	Inspect full simulation logs and account metas.

DEFINITION OF DONE

Treat this as your completion rubric, not just a nice-to-have list.

EXERCISE 1

01 `solana config get` points to devnet.

02 `anchor build`, `anchor deploy`, and `anchor test` succeed.

03 Program ID and test tx are visible on Explorer.

EXERCISE 2

01 Counter PDA `initialize` and `increment` paths reach expected value.

02 CLI account inspection confirms owner and data presence.

03 Duplicate `initialize` fails as expected.

04 Client-derived PDA matches on-chain PDA.

WHAT YOU LEAVE WITH

A working deployment workflow and your first PDA-based state program.

Operational confidence

Rust/CLI/Anchor stack verified and repeatable on devnet.

On-chain verification habit

You inspect program IDs, transactions, and account data instead of trusting terminal output alone.

PDA reasoning

You can derive deterministic addresses and debug seed mismatches.

AI review discipline

You check generated code for constraints, versions, and security assumptions.